

Components

React components are reusable UI units that handle their own logic, accept data through props, manage state, and efficiently update only the parts of the UI that change. In simpler terms: Components are like JavaScript functions. They accept arbitrary inputs (called “props”) and return React elements describing what should appear on the screen.

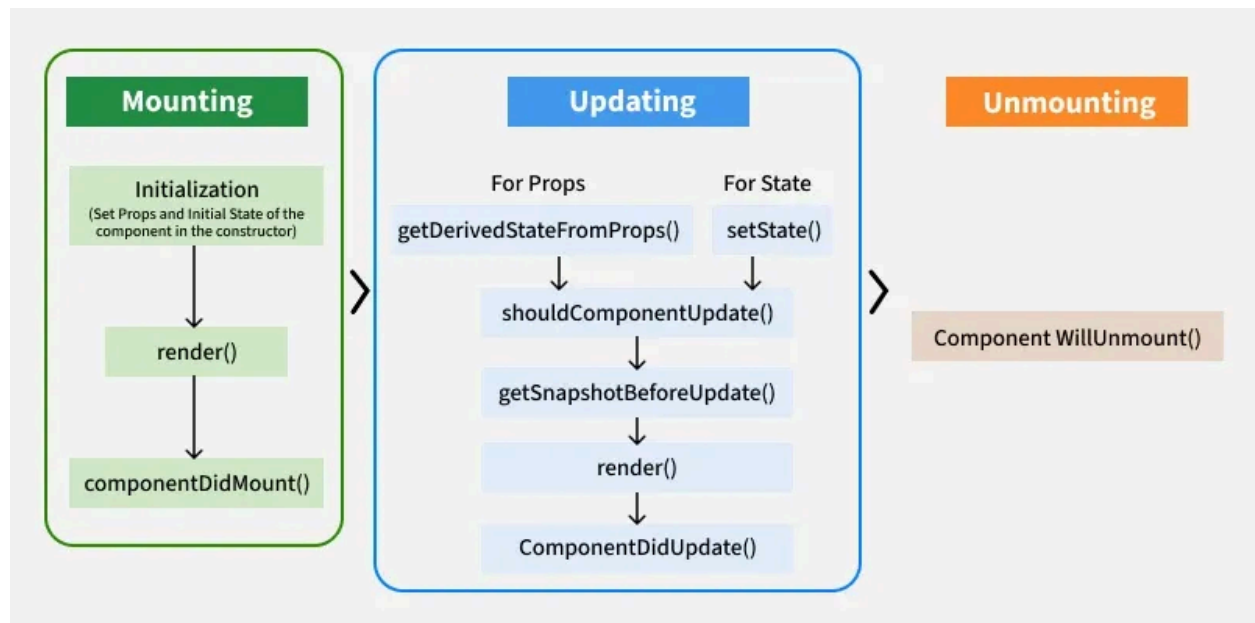
Syntax:

```
function ComponentName() {  
  return (  
    <JSX />  
  );  
}
```

```
import React from 'react';  
  
// Creating a simple functional component  
function Greeting() {  
  return (  
    <h1>Hello, welcome to React!</h1>  
  );  
}  
  
export default Greeting;
```

Component lifecycle

It refers to the sequence of events that happen from when a component is created, updated, and destroyed. In functional components (modern React), this is managed primarily through the **useEffect** hook.



Mounting

Mounting refers to the process of creating and inserting a component into the DOM **for the first time** in a React application. During mounting, React initializes the component, sets up its internal state (if any), and inserts it into the DOM.

- `constructor()`: Initializes the component's state and prepares it for rendering.
- `getDerivedStateFromProps(props, state)`: Used for updating the state based on props. Executed before every render.
- `render()`: Returns JSX that defines what should be displayed on the screen. Executes during every render phase to update the UI based on current state and props.
- `componentDidMount()` : This function is invoked right after the component is mounted on the DOM

Updating

Updating is the phase where a component re-renders in response to changes in its state or props, ensuring the UI reflects the latest data.

- `getDerivedStateFromProps()`: Compares incoming props with the current state to decide if state should be updated
- `setState()` :Updates the component's state explicitly and can be called at any time.
- `shouldComponentUpdate()`:
- `getSnapshotBeforeUpdate()`: is invoked just before the DOM is being rendered. It is used to store the previous values of the state after the DOM is updated.
- `componentDidUpdate()`:this function is invoked after the component is rendered

Unmounting

This is the final phase of the lifecycle of the component, which is the phase of unmounting the component from the DOM.

- `componentWillUnmount()`: This function is invoked before the component is finally unmounted from the DOM, i.e., this function gets invoked once before the component is removed from the page, and this denotes the end of the lifecycle

Types of React Components

Functional Components

Functional components are JavaScript functions that return React elements and are the preferred way to build modern React applications.

- Can **manage state and lifecycle logic** using React Hooks.
- Use a simpler syntax, making them ideal for reusable components.
- Offer better performance by avoiding the **use of the this keyword**.

```
function Greet(props) {  
  return <h1>Hello, {props.name}!</h1>;  
}
```

functional components don't have lifecycle methods. *useEffect* covers most of the needs. The rest are done by different hooks.

Class components

- Extends *React.Component* and **support state and lifecycle handling**.
- Manage component data using the **this.state** property.
- Use **lifecycle methods** such as **componentDidMount** and **componentDidUpdate**.

```
class Greet extends React.Component {  
  render() {  
    return <h1>Hello, {this.props.name}!</h1>;  
  }  
}
```

How to Pass Data to Components using Props

Props (properties) are **like instructions** you give to your components. They allow you to pass data from a **parent component to a child component**. This way, you can customize how components appear and behave. **In simple terms, props are like the parameters that you pass to a function.**

Once you have access to the data within the child component, you can use it to render dynamic content within the component's JSX. **This means you can display different UI elements based on the values of the props.**

```

JS App.js > ...
1  import React from 'react'
2  import Home from './Home'
3
4
5  function App() {
6    return (
7      <div>
8        <Home name="Andrew" game="coding" />
9      </div>
10   );
11 }
12
13 export default App;

```

```

JS Home.js > ...
1  import React from "react";
2
3  function Home(props) {
4    return (
5      <div id='home'>
6        <h1>{props.name} is my name.</h1>;
7        <h2> {props.game} is my game.</h2>;
8      </div>
9    );
10 }
11
12 export default Home;

```

But what if we wanted to make another component that is a child of Home. Let's make a component that displays the links to other sites like Twitter or Instagram.

The props that are passed down from App to its child (Home) are not naturally inherited by Home's child (Links).

We need to continue to have the parent pass props down to its child in order for the child to have access to those props.

```

JS App.js > ...
1  import React from 'react'
2  import Home from './Home'
3
4
5  function App() {
6    return (
7      <div>
8        <Home name='Andrew'
9          game='coding'
10         twitter='https://twitter.com/andrewasmit' />
11      </div>
12   );
13 }
14
15 export default App;

```

```

JS Home.js > ...
1  import React from "react";
2
3  function Home({name, game, twitter}) {
4    return (
5      <div id='home'>
6        <h1>{name} is my name.</h1>;
7        <h2> {game} is my game.</h2>;
8        <Links twitter={twitter}/>
9      </div>
10   );
11 }
12
13 export default Home;

```

```

JS Links.js > ...
1  import React from "react";
2
3  function Links(props) {
4    return (
5      <div id='home'>
6        <a href = {props.twitter}> Twitter </a>
7      </div>
8    );
9  }
10
11 export default Links;

```

Props.children

It allows us to write stuff in our **custom component**. This prop is very useful for "Compositing" components together, and can make JSX markup more intuitive or

reflective of the intended final structure of the DOM. Props.children can also be manipulated by the component.

```
1  import React from 'react'
2
3  function GreetComponent(props) {
4    return <div>
5      <h1>Hello {props.name}</h1>
6      {props.children}
7    </div>
8  }
9
10 function App() {
11   return (
12     <GreetComponent name="Mehul">
13       <p>DEF</p>
14     </GreetComponent>
15   )
16 }
17
18 export default App
```

Whatever we write in line inside line 13, it will be passed as children prop of *GreetComponent*.

JSX

JSX is a syntax extension for JavaScript. You can write HTML formatting inside a JavaScript file.

The Rules of JSX

1. Return a single root element-

To return elements from a component, wrap them with a single parent tag. You can use a `div` or `fragment` (`<></>`)

```
<div>
  <h1>Hedy Lamarr's Todos</h1>
  
</div>
```

```
<>
  <h1>Hedy Lamarr's Todos</h1>
  
</>
```

2. Close all the tags

3. `camelCase`

```


<button onClick={handleClick}>Click Me</button>
```

4. Passing the string attribute to JSX

```
export default function Avatar() {
  return (
    
  );
}
```

```
export default function Avatar() {
  const avatar = 'https://picsum.photos/200/300';
  const description = 'loempicsum';
  return (
    <img
      className="avatar"
      src={avatar}
      alt={description}
    />
  );
}
```

5. You can move several expressions into one object, and use them in your JSX inside curly braces

```

const person = {
  name: 'Gregorio Y. Zara',
  theme: {
    backgroundColor: 'red',
    color: 'yellow'
  }
};

export default function TodoList() {
  return (
    <div style={person.theme}>
      <h1>{person.name}'s Todos</h1>
    </div>
  );
}

```

HTML to JSX

```

1 <!-- Hello world -->
2 <div class="awesome" style="border: 1px solid red">
3   <label for="name">Enter your name: </label>
4   <input type="text" id="name" />
5 </div>
6 <p>Enter your HTML here</p>

```

```

1 <>
2   { /* Hello world */ }
3   <div className="awesome" style={{ border: "1px solid red" }}>
4     <label htmlFor="name">Enter your name: </label>
5     <input type="text" id="name" />
6   </div>
7   <p>Enter your HTML here</p>
8 </>
9

```

React routes

Check the code